LegalDataPlatform

Manual de uso sencillo · Plataforma ETL/ELT para datos legales y comerciales en Python + AWS + PostgreSQL

Para abrir el proyecto en tu navegador:
1) Levanta el API → `make api` (o `python -m uvicorn src.api.main:app --port 8000`)
2) Abre `http://127.0.0.1:8000/` · Swagger en `http://127.0.0.1:8000/docs`

¿Qué es esta plataforma?

LegalDataPlatform es una plataforma **ETL/ELT** que ingesta datos legales y comerciales desde múltiples fuentes (SEC EDGAR, GLEIF, CSV, APIs REST), los normaliza, valida con reglas de calidad y los carga en **PostgreSQL** particionado. Sigue arquitectura medallion **Bronze** → **Silver** → **Gold** y está pensada para correr local con Docker o en **AWS** (S3 + Lambda + Glue + ECS).

Capa	Qué contiene	Dónde vive
Bronze	Datos crudos tal como llegan	S3 (Parquet) · bucket `bronze`
Silver	Datos normalizados y con calidad validada	PostgreSQL (tablas particionadas)
Gold	Agregados de negocio listos para consumo	Materialized Views en PostgreSQL

1. Desarrollo de pipelines ETL/ELT en Python y AWS

Requisito del puesto: Diseñar y mantener pipelines ETL/ELT escalables en Python y AWS para la ingesta de datos de diversas fuentes legales y comerciales.

Se hizo eso ✓

Pipelines orquestados con **Prefect**, escritos en Python 3.11+. Cargas concurrentes con `asyncio`, transformaciones con **Polars** y **Pydantic**, despliegue en AWS vía Lambda + Glue + ECS. Cuatro flujos de ingesta listos: SEC EDGAR, GLEIF, CSV legal y comercial.

Dónde está cada componente

Componente	Ruta en el repo	Para qué sirve

Flujos orquestados (Prefect)	`src/pipelines/orchestration/`	Define los pipelines de extremo a extremo
Extractores de fuentes	`src/pipelines/extractors/`	SEC EDGAR, GLEIF, CSV, REST, S3, DB
Transformadores	`src/pipelines/transformers/`	Normalizers, enrichers, SCD2
Loaders	`src/pipelines/loaders/`	S3 Parquet (Bronze) y Postgres bulk (COPY)
AWS Lambda handlers	`src/aws/lambda_handlers/`	Triggers S3 → SQS → pipeline
AWS Glue jobs	`src/aws/glue_jobs/`	Normalización a escala con Spark
Cliente S3	`src/aws/s3/s3_client.py`	Wrapper boto3/aioboto3 para Bronze/Silver/Quarantine
Documentación detallada	`docs/PIPELINES.md` · `docs/AWS_DEPLOYMENT.md`	Diseño, deployment y trade-offs

Cómo correr un pipeline (paso a paso)

Paso 1. Levanta el stack local: `make up` (arranca Postgres, MinIO, Prefect, Grafana en Docker)

Paso 2. Aplica migraciones: `make migrate`

Paso 3. Corre un flujo. Tienes 4 disponibles:

- `make pipeline` — ingesta de documentos legales desde CSV
- `make sec-edgar` — ingesta desde la API real de SEC EDGAR
- `make gleif` — ingesta de identificadores LEI desde GLEIF
- `make commercial` — ingesta de transacciones comerciales

Paso 4. Observa el run en Prefect UI → `http://localhost:4200`

Paso 5. Verifica el archivo Parquet en MinIO → `http://localhost:9001` (bucket `bronze`)

Flujo end-to-end (ejemplo: documentos legales)

1. **Extract** — `CSVExtractor` lee en chunks de 5,000 filas con `asyncio` (no carga todo en memoria)
2. **Bronze** — `S3ParquetLoader` escribe Parquet particionado por fecha en S3/MinIO
3. **Normalize** — Pydantic valida cada registro contra `LegalDocumentSchema`

4. **Quality gate** — el motor de reglas YAML aplica validaciones declarativas
5. **Load** — `PostgresBulkLoader` usa `COPY` + `UPSERT` hacia tabla particionada
6. **Gold** — refresca Materialized Views `CONCURRENTLY` (sin bloquear lecturas)

En AWS

- **S3 PUT → Lambda** (`src/aws/lambda_handlers/s3_event_ingest.py`) encola un mensaje en **SQS**
- **SQS → Lambda** (`sqs_pipeline_trigger.py`) dispara el pipeline en **ECS**
- **AWS Glue** (`src/aws/glue_jobs/legal_doc_normalize.py`) escala normalización con Spark
- **Despliegue** documentado en `docs/AWS_DEPLOYMENT.md` e infra en `infra/`

2. Optimización de PostgreSQL

> **Requisito del puesto:** Gestionar y optimizar bases de datos PostgreSQL, garantizando alto rendimiento para datos relacionales complejos.

> **Se hizo eso ✓**
>
> Particionado por rango mensual, índices BRIN/GIN/expression/partial/covering, Materialized Views con refresh CONCURRENTLY, suite de benchmarks reproducibles y migraciones versionadas con Alembic. Documentado en `docs/POSTGRESQL_OPTIMIZATION.md`.

Dónde está cada optimización

Técnica	Archivo	Beneficio
Particionado mensual	`src/database/optimization/partitioning.sql`	Pruning automático + DROP O(1) + scans paralelos
Índices avanzados	`src/database/optimization/indexes_additional.sql`	BRIN, GIN, expresión, partial, covering
Materialized Views	`src/database/optimization/materialized_views.sql`	Agregados pre-calculados, refresh CONCURRENTLY
Helpers de query	`src/database/optimization/query_helpers.py`	EXPLAIN ANALYZE programático, profiling
Modelos ORM	`src/database/models/`	SQLAlchemy 2.0 async + dim/staging/legal/commercial
Migraciones	`src/database/migrations/versions/`	Alembic versionado, repetible, reversible
Benchmarks	`scripts/benchmarks/query_benchmark.py`	Mide p50/p95 antes y después de cambios

Optimizaciones aplicadas (lo que verás en producción)

- **Particionado RANGE mensual** en `legal_documents` y `transactions` — el planner ignora particiones irrelevantes y `VACUUM` / `REINDEX` trabajan por partición
- **Índice BRIN** en `transaction_date` → ~10KB/millón de filas vs ~30MB de un BTREE
- **Covering index** con `INCLUDE` para queries de "suma últimos 90 días" → index-only scans
- **Índices parciales** para contratos activos (filtra `WHERE status IN ('ACTIVE','PENDING')`)
- **Índices GIN** sobre columnas JSONB (`tags`, `metadata`) para búsquedas por tag/atributo
- **Índice de expresión** `lower(legal_name)` para búsquedas case-insensitive
- **Materialized Views** precomputan revenue mensual por contraparte; refresh `CONCURRENTLY` no bloquea lectores
- **UPSERT vía** `COPY` **+ tabla staging** en lugar de `INSERT` fila por fila → 10–50× más rápido

Cómo medir performance

Paso 1. Ejecuta los benchmarks: `make benchmark`

Paso 2. Captura evidencia con `make evidence` → genera `docs/evidence/benchmarks.md`

Paso 3. Revisa Grafana en `http://localhost:3000` para ver latencia de queries en tiempo real

3. Calidad y normalización de datos (Fuente de verdad)

Requisito del puesto: Implementar procesos rigurosos de validación y limpieza que mantengan los estándares de "Fuente de verdad".

Se hizo eso ✓

Doble capa de validación: (1) Pydantic para tipado y normalización, (2) reglas YAML declarativas con severidades `error` / `warning`. Registros que fallan reglas `error` bloquean el pipeline y van a un bucket de cuarentena en S3 con el motivo. Integración opcional con Great Expectations.

Dónde está cada control

Control	Archivo	Qué hace
Reglas declarativas YAML	`src/data_quality/rules/legal_documents.yaml` `src/data_quality/rules/transactions.yaml`	Define rules sin código (not_null, regex, in_set, row_count_range)
Motor de reglas	`src/data_quality/validators/rule_engine.py`	Aplica las reglas y separa severidad error vs warning

Schemas Pydantic legal	`src/schemas/legal.py`	LegalDocumentSchema, LegalEntitySchema, RegulationSchema
Schemas Pydantic comercial	`src/schemas/commercial.py`	TransactionSchema, ContractSchema, CounterpartySchema
Normalizers	`src/pipelines/transformers/normalizers.py`	Aplica schema y separa válidos/rechazados
SCD2	`src/pipelines/transformers/scd2.py`	Slowly Changing Dimensions tipo 2 para historizar cambios
Integración Great Expectations	`src/data_quality/ge_integration.py`	Suites adicionales para auditoría
Reportes de DQ	`src/data_quality/reports/`	Output de cada corrida (válidos, rechazos, warnings)
Documentación	`docs/DATA_QUALITY.md`	Filosofía, severidades, runbooks de incidentes

Ejemplo: reglas activas para `legal_documents`

Regla	Tipo	Severidad
document_id_not_null	not_null sobre source_id	error (bloquea)
document_hash_is_sha256	regex `^[a-f0-9]{64}$`	error (bloquea)
document_type_in_taxonomy	in_set (JUDGMENT, REGULATION, CONTRACT, NOTICE, FILING, DECREE)	warning (alerta)
jurisdiction_not_null	not_null	error (bloquea)
row_count_reasonable	row_count_range (1 ≤ N ≤ 10M)	error (bloquea)

Cómo se aplica el principio "Fuente de verdad"

1. Cada registro entra a **Bronze** tal cual (auditoría completa)
2. Pydantic intenta normalizarlo. Si falla → va a **Quarantine** con el error
3. El motor de reglas YAML aplica controles. Reglas `error` → bloquean el flow vía `assert_no_errors()`

4. Reglas `warning` → emiten métricas Prometheus pero no bloquean
5. Solo registros que pasan ambas capas llegan a **Silver** (la "fuente de verdad")
6. Cada registro lleva un `source_hash` SHA-256 para idempotencia y trazabilidad

Cómo agregar una nueva regla (sin tocar código)

Paso 1. Edita `src/data_quality/rules/legal_documents.yaml`

Paso 2. Agrega un bloque como:

```
- name: title_min_length
  type: min_length
  column: title
  params: { min: 5 }
  severity: warning
```

Paso 3. Re-ejecuta el pipeline. La regla se aplicará automáticamente.

Resumen de URLs y comandos

Servicio	URL / Comando
API + Dashboard	`http://127.0.0.1:8000/`
Swagger docs	`http://127.0.0.1:8000/docs`
Prefect UI	`http://localhost:4200`
MinIO (S3 local)	`http://localhost:9001`
Grafana	`http://localhost:3000`
Levantar todo	`make up`
Migrar BD	`make migrate`
Correr pipeline	`make pipeline` · `make sec-edgar` · `make gleif` · `make commercial`
Levantar API	`make api`

Tests	`make test`
Benchmark + evidencia	`make benchmark` · `make evidence`